

NAHDA University Faculty of Computer science Dep.:Computer science	Academic level: 4 Semester: 1 Course code : CS401 Corse title: Expert system Instructor: Dr.Rabab Hamed Sheet 2	
---	---	---

Objectives:

- ✓ What is the meaning of knowledge?
- ✓ The main benefits of Regressions.
- ✓ Regression - How to program the Best Fit Slope.
- ✓ Mean-Shift Clustering
- ✓ The analysis of data model using kernel method

Software requirements

- ✓ Anaconda navigator software

Pre lab

- ✓ Main types of expert methods

Note :

The main library of neural in python

1. Pandas
2. Numpy
3. matplotlib
4. Tensorflow 2
5. Scikit-Learn
6. Keras

Note that:

Linear regression analysis is used to predict the value of a variable based on the value of another variable. The variable you want to predict is called the dependent variable. The variable you are using to predict the other variable's value is called the independent variable

In order to begin designing a model, it is necessary to possess a comprehensive understanding of regression and its corresponding advantages for the model.

Example1: Regression - How to program the Best Fit Slope.

$$m = \frac{\bar{x} \cdot \bar{y} - \overline{xy}}{(\bar{x})^2 - \overline{x^2}}$$

The best fit line equation is :

$$b = \bar{y} - m\bar{x}$$

Part1 linear regression for best fit:-

```
from statistics import mean
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import style

xs = np.array([1,2,3,4,5], dtype=np.float64)
ys = np.array([5,4,6,5,6], dtype=np.float64)

def best_fit_slope_and_intercept(xs,ys):
    m = (((mean(xs)*mean(ys)) - mean(xs*ys)) /
          ((mean(xs)*mean(xs)) - mean(xs*xs)))

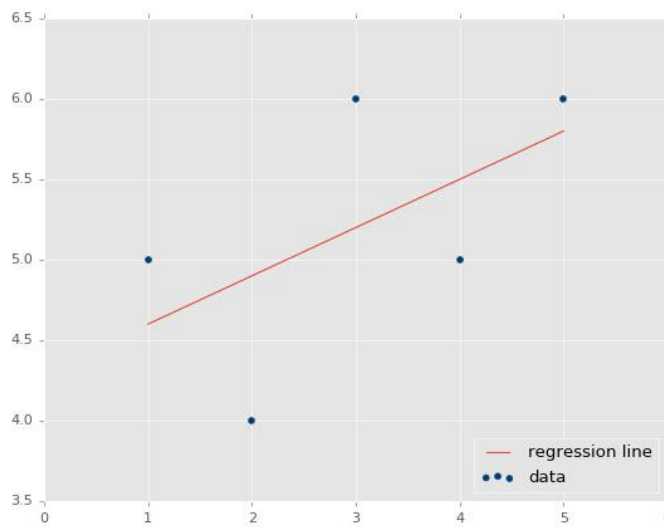
    b = mean(ys) - m*mean(xs)

    return m, b

m, b = best_fit_slope_and_intercept(xs,ys)

print(m,b)
regression_line = [(m*x)+b for x in xs]
regression_line = []
for x in xs:
    regression_line.append((m*x)+b)
style.use('ggplot')
plt.scatter(xs,ys,color='#003F72')
plt.plot(xs, regression_line)
plt.show()
```

Output:



Now You have your model based on regression, you just fill in x =7. and predict out a couple of points:

```
predict_x = 7
predict_y = (m*predict_x)+b
print(predict_y)
plt.scatter(xs,ys,color='#003F72',label='data')
plt.plot(xs, regression_line, label='regression line')
plt.legend(loc=4)
plt.show()
```

Part2 Mean-Shift Clustering:

Mean shift is falling under the category of a clustering algorithm in contrast of Unsupervised learning that assigns the data points to the clusters iteratively by shifting points towards the mode (mode is the highest density of data points in the region, in the context of the Mean shift). As such, it is also known as the Mode-seeking algorithm. Mean-shift algorithm has applications in the field of image processing and computer vision. Mean-shift clustering is a non-parametric, density-based clustering algorithm that can be used to identify clusters in a dataset. It is particularly useful for datasets where the clusters have arbitrary shapes and are not well-separated by linear boundaries.

The process of mean-shift clustering algorithm can be summarized as follows:

- Initialize the data points as cluster centroids.
- Repeat the following steps until convergence or a maximum number of iterations is reached:

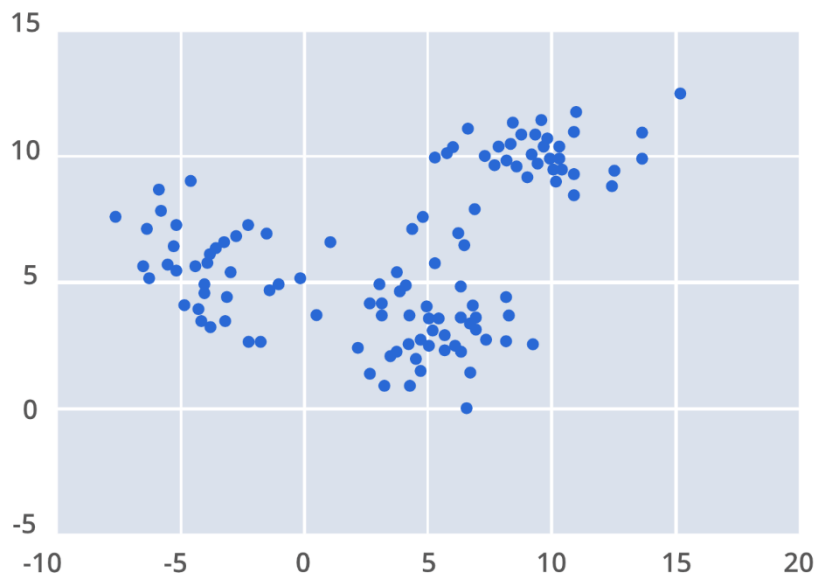
For each data point, calculate the mean of all points within a certain radius (i.e., the “kernel”) centered at the data point.

- Shift the data point to the mean.
- Identify the cluster centroids as the points that have not moved after convergence.
- Return the final cluster centroids and the assignments of data points to clusters.

One of the main advantages of mean-shift clustering is that it does not require the number of clusters to be specified beforehand. It also does not make any assumptions about the distribution of the data, and can handle arbitrary shapes and sizes of clusters. However, it can be sensitive to the choice of kernel and the radius of the kernel.

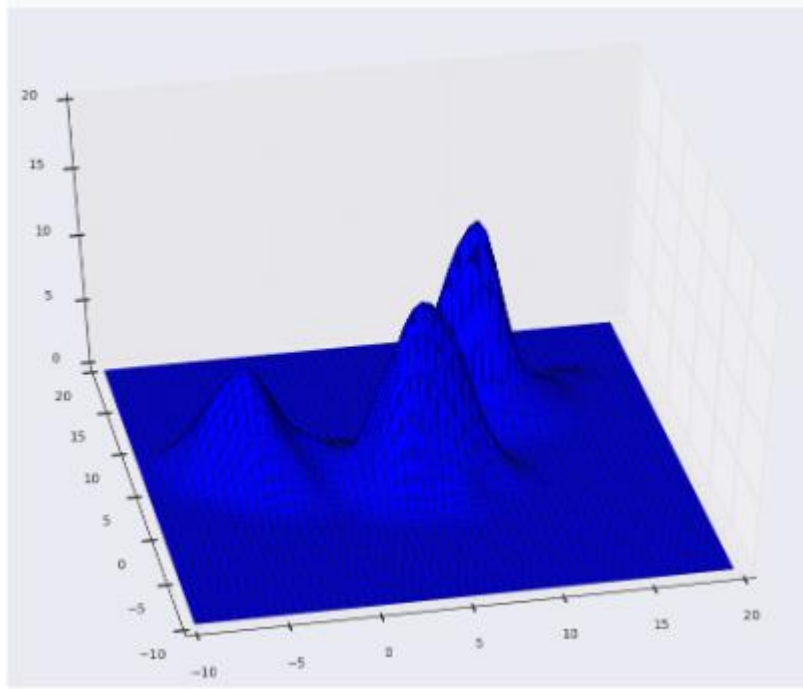
Mean-Shift clustering can be applied to various types of data, including image and video processing, object tracking and bioinformatics.

Kernel Density Estimation

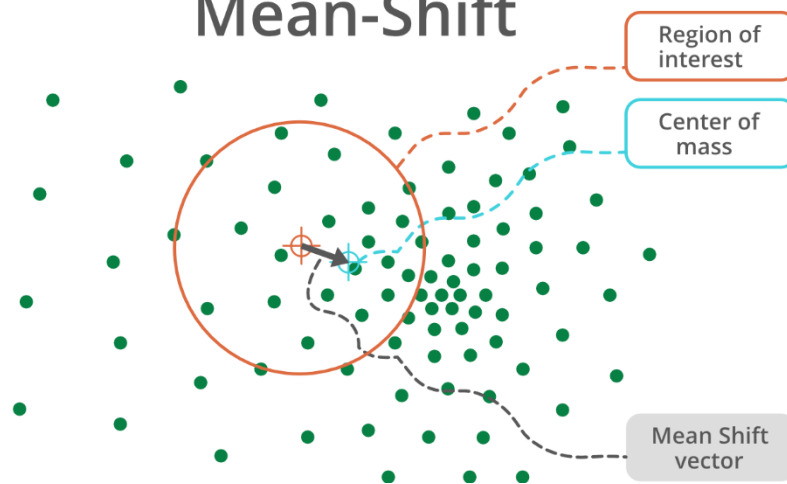


Mean-shift builds upon the concept of Kernel Density Estimation(KDE), in short KDE. Imagine that the above data (see Figure)was sampled from a probability distribution. KDE is a method to estimate the underlying distribution also called the probability density function for a set of data. It works by placing a kernel on each point in the data set. A kernel is a fancy mathematical word for a weighting function generally used in convolution. There are many different types of kernels, but the most popular one is the Gaussian kernel. Adding up all of the individual kernels generates a probability surface example density function. Depending on the kernel bandwidth parameter used, the resultant density function will vary.

Below is the KDE surface for our points above using a Gaussian kernel with a kernel bandwidth of 2.



Mean-Shift



Example:

we begin in this sheet by building our own mean shift algorithm from scratch. To begin, we will start with some code from part 37 of this series, which was when we began building our custom K Means algorithm. We will add one more cluster/group to the original data. Feel free to add the new data or leave it the same as it was.

```
import matplotlib.pyplot as plt
from matplotlib import style
style.use('ggplot')
import numpy as np

X = np.array([[1, 2],
              [1.5, 1.8],
              [5, 8 ]],
```



```
        break

    self.centroids = centroids

clf = Mean_Shift()
clf.fit(X)

centroids = clf.centroids

plt.scatter(X[:,0], X[:,1], s=150)

for c in centroids:
    plt.scatter(centroids[c][0], centroids[c][1], color='k',
marker='*', s=150)

plt.show()
```

References :

- 1) <https://pythonprogramming.net/how-to-program-best-fit-line-machine-learning-tutorial/?completed=/how-to-program-best-fit-line-slope-machine-learning-tutorial/>
- 2) <https://pythonprogramming.net/mean-shift-from-scratch-python-machine-learning-tutorial/>
- 3) <https://www.geeksforgeeks.org/ml-mean-shift-clustering/>